

Lösningsgenomgång

Chalmers Challenge 2023

Pub Crawl

- Add one to the queue length (Fritjof himself counts as a person in the queue)
- Multiply by the amount of time each person needs to wait
 - Queue time = $(k+1)*t$
- Select the pub with the maximum queue time, breaking ties by picking earlier

chalmerschallenge23_pubbrunda.py

```
1 maxbar = ""
2 maxtime = 0
3 for i in range(int(input(""))):
4     line = input("").split(" ")
5     bartime = (int(line[1])+1)*int(line[2])
6     if bartime > maxtime:
7         maxtime = bartime
8         maxbar = line[0]
9 print(maxbar + " " + str(maxtime))
```

Solution by Vilgot J

Memory Game

- If two cards with the same number are seen, they are definitely a pair
- If we can take N-1 pairs we can also take the last pair

```
n, k = [int(i) for i in input().split()]
nums = input().split()

ans = len(nums) - len(set(nums))

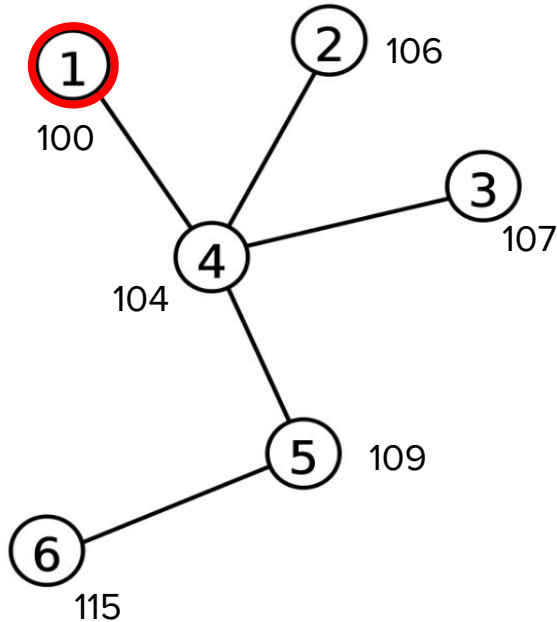
if (n - ans) == 1:
    ans = n

print(ans)
```

Solution by Mateusz Drwal

Gasqutvå subtask 1 ($N \leq 100$)

Run a DFS from each student to calculate the donations if that student starts.



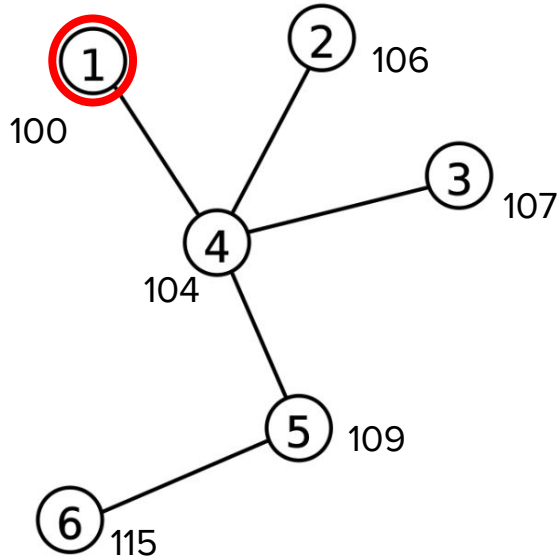
$$100 + 104 + 106 + 107 + 109 + 115 = 641$$

$$O(N^2)$$

Gasqutvå subtask 2 ($N \leq 10^5$)

Keep track of the number of students on each side of each edge.

Move around the graph and update the answer with each step.

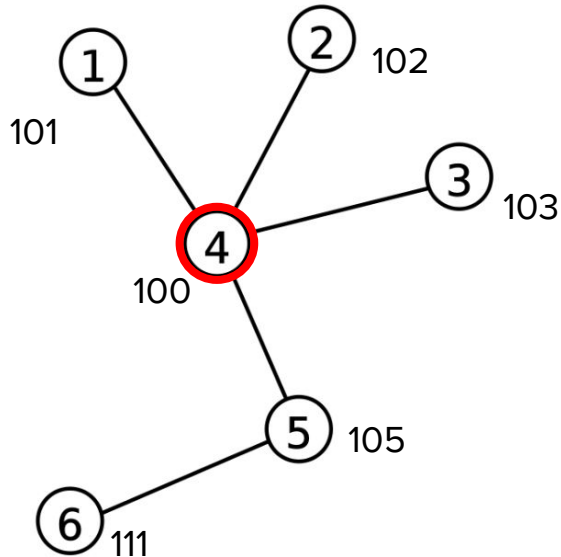


Answer: 641

Gasqutvå subtask 2 ($N \leq 10^5$)

Keep track of the number of students on each side of each edge.

Move around the graph and update the answer with each step.



Answer: 641

Five nodes decrease by 4, one increase by 1.

New answer: $641 - 5 \cdot 4 + 1 \cdot 1 = 622$

$O(N)$

Gasqutvå subtask 2 ($N \leq 10^5$)

```
17
18 ll n;
19 vi aa, cnt, score, vis, _;
20 vector<vi> es;
21
22 ll cnt_nodes(ll v){
23     trav(u, es[v]){
24         if (vis[u] == 0){
25             vis[u] = 1;
26             cnt[v] += cnt_nodes(u);
27         }
28     }
29     return cnt[v] + 1;
30 }
31
32 void scores(ll v){
33     trav(u, es[v]){
34         if (vis[u] == 0){
35             vis[u] = 1;
36             score[u] = score[v] + aa[v]*(n - cnt[u] - 1) - aa[u]*(cnt[u] + 1);
37             scores(u);
38         }
39     }
40 }
41
```

Solution by Hugo Eberhard

```
42 int main(){
43     ios::sync_with_stdio(0);
44     cin.tie(NULL); cout.tie(NULL);
45     cout << setprecision(15) << fixed;
46     cin >> n;
47     aa.assign(n, 0);
48     cnt.assign(n, 0);
49     score.assign(n, 0);
50     es.assign(n, _);
51     rep(i, 0, n){
52         cin >> aa[i];
53     }
54     rep(i, 0, n-1){
55         ll a, b;
56         cin >> a >> b;
57         es[a-1].push_back(b-1);
58         es[b-1].push_back(a-1);
59     }
60
61     vis.assign(n, 0);
62     vis[0] = 1;
63     cnt_nodes(0);
64
65     vis.assign(n, 0);
66     vis[0] = 1;
67     scores(0);
68
69     ll ans = 0, person = 0;
70     rep(i, 0, n){
71         if (score[i] > ans){
72             ans = score[i];
73             person = i;
74         }
75     }
76     cout << person + 1 << endl;
77 }
78
79
```

Exponentially Fun Problem

- $S(12) = S(2*2*3) = 2+2+3 = 7$. Problem: Find smallest and largest X satisfying $S(X) = N$
- **Maximum:** $S(9) = S(3*3) = 6$, $S(8) = S(2*2*2) = 6$
- Maybe these? $S(3^n) = 3n$ $S(3^n * 2) = 3n+2$ $S(3^n * 4) = 3n+4$
- Proof by $3*(p-3) > p$ for large p , check small cases manually.
- **Minimum:** We want all kinds of x . Consider for example $S(p*2) = p+2$ with p being some large prime. It turns out we cannot do this greedily (although it works on most random numbers!). Instead we do dynamic programming.
- $DP[n]$ = smallest solution x to $S(x) = n$. Try every prime and compute $DP[n]$ from
- $DP[n-2]$, $DP[n-3]$, $DP[n-5]$, $DP[n-7]$ and so on for every prime. Start by sieving.


```

1 const readline = require('readline');
2
3 const rl = readline.createInterface({
4   input: process.stdin,
5   output: process.stdout
6 });
7
8 let dp = {};
9
10 let primes = [];
11 for (let i = 2; i <= 30000; i++) {
12   let is_prime = true;
13   for (let p of primes) {
14     if (i % p === 0) {
15       is_prime = false;
16       break;
17     }
18   }
19   if (is_prime) primes.push(i);
20 }
21

```

```

22 function f(n) {
23   if (n === 0) return 1;
24
25   if (n in dp) {
26     return dp[n];
27   }
28
29   let min = 30000 ** 2;
30
31   let min_p = [];
32
33   for (let p of primes) {
34     if (p > n) break;
35
36     let l_min = f(n - p);
37     l_min *= p;
38     if (l_min < min) min = l_min;
39   }
40
41   //console.log(n, min_p);
42
43   dp[n] = min;
44   return dp[n];
45 }

```

```

47 rl.on('line', async (line) => {
48   let n = parseInt(line);
49   for (let i = 0; i < n; i++) f(i);
50
51   let tmp = n;
52   let max = 1;
53   while (tmp >= 2) {
54     if (tmp === 4 || tmp === 2) {
55       tmp -= 2;
56       max *= 2;
57     } else {
58       tmp -= 3;
59       max *= 3;
60     }
61     max %= 1000000007;
62   }
63
64   let min = f(n);
65   console.log((min % 1000000007).toString() + " " + max);
66   process.exit();
67 });

```

Solution by Carl Sundvall

Exponentially Fun Problem (unintentional)

```
1 /sIuePyI5y3dNZ9uNLzq8lJQ0b0qKTAUe+gqpnU0M+o0dK+Z00TIEpU0W1PLTKYyVVNNHTK/:  
2 /BAsQXJfQCD7zHW9oyTtjzSsCJL9WDLy19oFsPPpyTSRVm7+LAs087MwVmXJ3Ywzq6MeUieZ:  
3 /SBD9XtMR2WxnkXe0uEYD2Fd77hemo0vx1Ch7aG+41nWLCXrnSmVBh265Ap9X6SgfVJS5AHRj  
4 /dKy7sCEjSLvt+xVp03I/NN98sVE3g5cMqnpDg6Vt+sRHWH  
5 /FQJqeEI5q0kCTNBnyRFqaQs7GvmRJtHs0+j57L37z8N07QgG5AgL4AkDE81cKNxZiju0z8d:  
6 //rH3gAcu9JjnySynzwwbIRhT/n3Rw2w6QwcyjMvI/hyUcIz1ma0ZZm7BTdRAeT/ObQotq3  
7 /A4QyU5h9ACbTXzNaXIqzG+GRYQtRHuLN0mVzvZ02kGa  
8 /SC90RDyUjfs+A8zMg501r6LzxNNdkzkIZCYcMN001BCDStQnssdDKWbIN0KsRgE3I4DyjiWl  
9 /IfjNv7qTKdYvvTxNH15ZSPPDZm2y2  
10 /2JfE/lkFKTIQXyasdnfBS+zLaxHwUgUcksKvdcB3jlkjbiATbkoZ1eCuezGWod0PzqwJ5aM:  
11 lu = eval(zlib.decompress(base64.b64decode(data)))  
12  
13 MOD = 10**9 + 7  
14  
15 x = int(input())  
16 smallest = lu[x] % MOD  
17  
18 largest = 1  
19 while x > 4:  
20     x -= 3  
21     largest = (largest * 3) % MOD  
22  
23 if x == 4:  
24     largest = (largest * 2 * 2) % MOD  
25 elif x == 3:  
26     largest = (largest * 3) % MOD  
27 elif x == 2:  
28     largest = (largest * 2) % MOD  
29  
30 print(smallest, largest)
```

Jonathan L

Based Porridge

- We have N students, N servings of porridge of K different kinds. Each pair (student, kind) has happiness score. Need to distribute the servings, minimizing $\max_happiness - \min_happiness$ (equality yay!)
 - Some happiness intervals (min, max) are possible, some are not.
 - Tightest *max* given *min* is a monotonic function (it does not work, until it does, and it continues working). $\Rightarrow$ We sweep *max* while iterating *min*.
-
- Is it possible to assign within an interval $[\min_happiness, \max_happiness]$?
 - This is asking to find a perfect matching in a bipartite graph of students and servings. Solved by flow, particularly hopcroft-karp

Based Porridge

```
11 bool dfs(int a, int L, vector<vi>& g, vi& btoa, vi& A, vi& B) {
12     if (A[a] != L) return 0;
13     A[a] = -1;
14     for (int b : g[a]) if (B[b] == L + 1) {
15         B[b] = 0;
16         if (btoa[b] == -1 || dfs(btoa[b], L + 1, g, btoa, A, B))
17             return btoa[b] = a, 1;
18     }
19     return 0;
20 }
21 }
22
23 int hopcroftKarp(vector<vi>& g, vi& btoa) {
24     int res = 0;
25     vi A(g.size()), B(btoa.size()), cur, next;
26     for (;;) {
27         fill(all(A), 0);
28         fill(all(B), 0);
29         /// Find the starting nodes for BFS (i.e. layer 0).
30         cur.clear();
31         for (int a : btoa) if (a != -1) A[a] = -1;
32         rep(a, 0, sz(g)) if (A[a] == 0) cur.push_back(a);
33         /// Find all layers using bfs.
34         for (int lay = 1;; lay++) {
35             bool islast = 0;
36             next.clear();
37             for (int a : cur) for (int b : g[a]) {
38                 if (btoa[b] == -1) {
39                     B[b] = lay;
40                     islast = 1;
41                 }
42                 else if (btoa[b] != a && !B[b]) {
43                     B[b] = lay;
44                     next.push_back(btoa[b]);
45                 }
46             }
47             if (islast) break;
48             if (next.empty()) return res;
49             for (int a : next) A[a] = lay;
50             cur.swap(next);
51         }
52         /// Use DFS to scan for augmenting paths.
53         rep(a, 0, sz(g))
54             res += dfs(a, 0, g, btoa, A, B);
55     }
56 }
57 }
```

Standard
KACTL

```
58 int N, K;
59 vector<int> a;
60 vector<vector<int>> h;
61
62 bool is_possible(int h_lo, int h_hi) {
63     vector<vi> g(N);
64     rep(i, 0, N) {
65         int right_ind = 0;
66         rep(j, 0, K) {
67             rep(l, 0, a[j]) {
68                 if (h_lo <= h[i][j] && h[i][j] <= h_hi) {
69                     g[i].push_back(right_ind);
70                 }
71                 right_ind++;
72             }
73         }
74     }
75     vi btoa(N, -1);
76     int matching = hopcroftKarp(g, btoa);
77     return matching == N;
78 }
79
80 int main() {
81     cin.tie(0) -> sync_with_stdio(0);
82     cin.exceptions(cin.failbit);
83     cin >> N >> K;
84     a.resize(K);
85     rep(i, 0, K) {
86         cin >> a[i];
87     }
88     h = vector<vector<int>>(N, vector<int>(K));
89     set<int> hs;
90     rep(i, 0, N) {
91         rep(j, 0, K) {
92             cin >> h[i][j];
93             hs.insert(h[i][j]);
94         }
95     }
96     vector<int> hs_vec(all(hs));
97     int ans = 2e9;
98     int lo = 0, hi = 0;
99     while (lo < sz(hs_vec)) {
100         while (hi < sz(hs_vec) && !is_possible(hs_vec[lo], hs_vec[hi])) {
101             hi++;
102         }
103         if (hi < sz(hs_vec)) {
104             ans = min(ans, hs_vec[hi] - hs_vec[lo]);
105         }
106         lo++;
107     }
108     cout << ans << endl;
109 }
110 }
```

A little bit of
sweeping

Solution by Mårten Wiman

Negative Graph

The problem: To find the shortest path between two nodes in a weighted directed graph. Some edges have negative weights. $N \leq 10^5$ nodes and edges, only $K \leq 50$ negative edges.

In the absence of negative weights we can find shortest paths with Dijkstra.
 $O(N \log N)$

With many negative weights we can do Bellman-Ford (N rounds of every node updating the distance of all downstream nodes) in $O(VE)$

But Bellman-Ford is too slow here! $O(N^2)$

Negative Graph

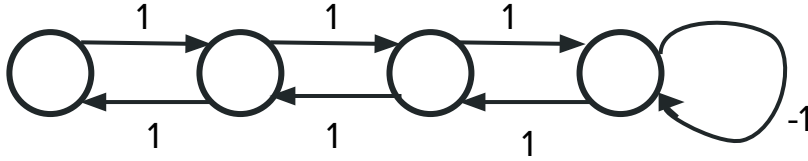
Very few edges are negative. Let's start by reducing the graph!

1. The start node, end node and nodes connected to a negative edge are *interesting*.
2. Remove negative edges
3. Find the shortest distance between all pairs of interesting nodes with dijkstra (no negative edges)
4. Create a graph with only the interesting nodes, shortest paths and negative edges.
5. Run Bellman-Ford on this graph.

$O(K)$ nodes, but this graph is dense. $O(K^2)$ edge. Total time complexity of $O(K^3)$ (but this is fine)

Negative Graph

Most common WA:



Negative Graph